3. D_{KL} 用来表示两个分部的相似程度.

① PDF $P(\underline{x})$ unknown.

$$\hat{P}(\underline{x}) = \frac{1}{N}\sum_{n=1}^{N} k(\underline{x}-\underline{x}_{(n)})$$

kernel $k(\underline{x})$: a PDF-like function.

$k(\underline{x}) \geq 0$, $\int k(\underline{x}) = 1$

② $$D_{KL}(P\|q) = \int P(\underline{x}) \ln \frac{P(\underline{x})}{q(\underline{x})} d\underline{x} = E_{\underline{x}\sim P(\underline{x})}\left[\ln \frac{P(\underline{x})}{q(\underline{x})}\right]$$

1. $D_{KL}(P\|q) \geq 0$.
   在 $P(x) = q(x)$ 时取等.
2. $D_{KL}(P\|q) \neq D_{KL}(q\|P)$
3. $D_{KL}(P\|q) = D_{KL}(P_1\|q_1) + D_{KL}(P_2\|q_2)$
4. $$D_{KL}(P\|q) = \int p \ln \frac{p}{q} dx$$
$$= \int p \ln p \, dx - \int p \ln q \, dx$$
$$= -H(p) + H(p,q)$$

在 Given $P(\underline{x})$, $\rightarrow H(p)$ fixed.

$$\min_q D_{KL}(P\|q) \hat{=} \min_q H(P,q)$$

KLD CE.

DL 是在做什么?

*) $P(\underline{x}|\underline{y})$, $P(\underline{y})$ unknown $\longrightarrow$ $P(\underline{y}|\underline{x})$ unknown.

*) 我们要近似一个 $\hat{P}(\underline{y}|\underline{x})$

此时使用 $q(\underline{y}|\underline{x};\theta)$, 通过设计 DNN 得到.

$\underline{x} \longrightarrow$ [DNN, $\underline{\theta}$] $\longrightarrow \hat{\underline{y}}$

实现这一过程.

△) $q(\cdots)$ 是 DNN 的结构. (已知)

△) $\underline{\theta}$ 是未知的, learned from data.

$$\min_{\underline{\theta}} D_{KL}(P(\underline{y}|\underline{x}) \| q(\underline{y}|\underline{x};\theta))$$

用这样的标准优化.

$$q(x,y;\theta) = q(\underline{y}|\underline{x};\theta) \cdot q(\underline{x})$$

对于离散的有 PMF.

常用分布. categorial distribution

$$\begin{cases} P(\underline{x}) = \prod_{i=1}^{c} p_i^{x_i} \\ \ln P(\underline{x}) = \sum_{i=1}^{c} x_i \ln p_i = \underline{x}^T \cdot \ln \underline{p} \end{cases}$$

那我们可以进一步简化.

$$H(p(x,y), q(x,y;\theta)) = -\int p(x,y) \ln q(x,y;\theta) \, d\underline{x} \, d\underline{y}.$$

$$= -\int p(x,y) \cdot \ln q(\underline{y}|\underline{x};\theta) \, dx dy - \underbrace{\int p(\underline{x},\underline{y}) \cdot \ln q(\underline{x}) dx dy}_{\text{const}}$$

only $\underline{\theta}$ unknown.

$$= \int p(x,y) \underbrace{[-\ln q(\underline{y}|\underline{x};\theta)]}_{\text{★ } L(\underline{x},\underline{y},\underline{\theta}) \text{ : loss for } (\underline{x},\underline{y})} dx dy + \text{const}$$

如果 $\hat{p}(\underline{x},\underline{y}) = \frac{1}{N}\sum_{n=1}^{N} \delta(\underline{x}-\underline{x}_{(n)}, \underline{y}-\underline{y}_{(n)})$

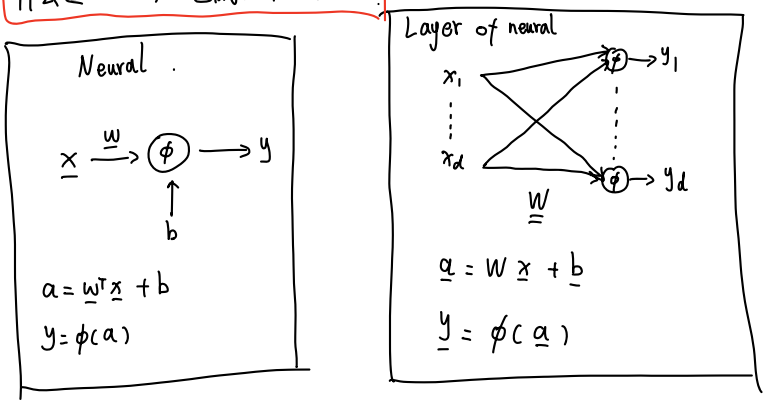
被我们这样估计.

这是一种分布 $\delta(\underline{x}) = \begin{cases} \infty & \underline{x}=0 \\ 0 & \underline{x}\neq 0 \end{cases}$

*) $\int \delta(\underline{x}) d\underline{x} = 0$ *) $\int \delta(\underline{x}-\underline{x}_0) f(\underline{x}) d\underline{x} = f(\underline{x}_0)$

$$H(\hat{p},q) = \text{const} + \underbrace{\frac{1}{N}\sum_{n=1}^{N} L(\underline{x}_{(n)}, \underline{y}_{(n)};\underline{\theta})}_{\text{★ } L(\underline{\theta}) \text{ cost function}}$$

什么是 DNN，他的结构是什么

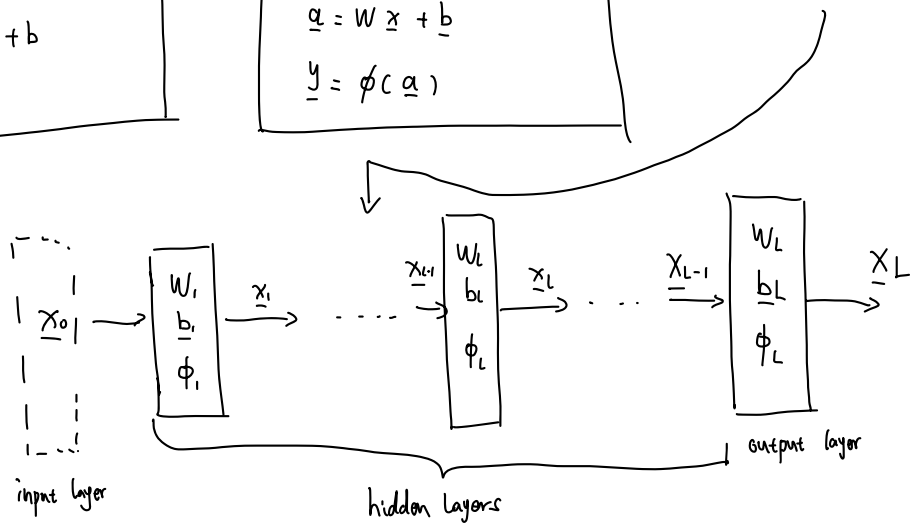


dense layer.

each input connected with each neural.

DNN.

every layer is dense layer.



$\underline{x}_0 \in \mathbb{R}^{M_0}$ $\underline{x}_L \in \mathbb{R}^{M_L}$.

Network.

$\underline{x}_L = f(\underline{x}_0; \underline{\theta})$

$$\underline{\theta} = \begin{bmatrix} \mathrm{vec}(\underline{W}_1) \\ \vdots \\ \mathrm{vec}(\underline{W}_L) \end{bmatrix} \in \mathbb{R}^{N_P}.$$

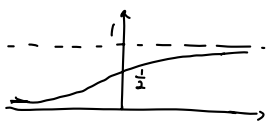
Number of parameter $N_P = \sum_{l=1}^{L} M_l (M_{l-1} + 1)$

// multiplication $N_x = \sum_{l=1}^{L} M_l M_{l-1}$

下面我们介绍一些 activation function. (ϕ).

① sigmoid.

$$\phi(a) = \sigma(a) = \frac{1}{1+e^{-a}}$$



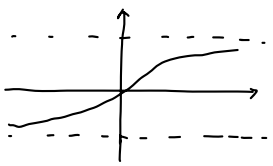
$$\phi(-a) = 1 - \phi(a)$$

$$\frac{d\phi}{da} = \phi(a)[1-\phi(a)] = \phi(a)\phi(-a)$$

② hyperbolic tangent (tanh)

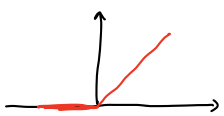
$$\phi(a) = \tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}$$

$$= 2\sigma(a) - 1$$



③ ReLU

$$\phi(a) = \mathrm{ReLU}(a) = \begin{cases} a & a > 0 \\ 0 & a < 0 \end{cases} = \max(a, 0)$$



④ softmax.

$\phi(\underline{a})$: $\underline{a} = [a_i] \in \mathbb{R}^C \longrightarrow \mathbb{R}^C$

$$\phi(\underline{a}) = \mathrm{softmax}(\underline{a}) = \begin{bmatrix} \phi_1(\underline{a}) \\ \vdots \\ \phi_C(\underline{a}) \end{bmatrix}$$

$$\phi_i(\underline{a}) = \frac{e^{a_i}}{\sum_{j=1}^{C} e^{a_j}} \in (0, 1).$$

$$\frac{\partial \phi_i(\underline{a})}{\partial a_j} = \cdots = \begin{cases} \phi_i(\underline{a}) \cdot [1 - \phi_i(\underline{a})] & i = j \\ -\phi_i(\underline{a})\,\phi_j(\underline{a}) & i \neq j \end{cases}$$

for $C = 2$

$$\phi_1(\underline{a}) = \frac{e^{a_1}}{e^{a_1} + e^{a_2}} = \sigma(a_1 - a_2)$$

$$\phi_2(\underline{a}) = 1 - \phi_1(\underline{a}) = \sigma(a_2 - a_1)$$

Model learning.

Given

① Training dataset.

② a DNN $f(x;\theta)$.

⟶ design loss function for different task.

learn θ from data.

For **regression**.

$x \in \mathbb{R}^d$ input

$y \in \mathbb{R}^d$ output.

$y = f(x;\theta) + z$, z: noise.

我们已知 z 的分布 可推导 y 的分布.

e.g. A. $z \sim N(0, \sigma I)$

$\Rightarrow y \sim N(f(x;\theta), \sigma I)$

$\|$

$y|x$

$l(x,y;\theta) = -\ln q = \text{const} + \frac{1}{2\sigma^2}\|y - f(x;\theta)\|^2$

$L(\theta) = \frac{1}{N}\sum_{n=1}^{N} \|y^{(n)} - f(x^{(n)};\theta)\|$ (l_2: ✓, norm: ✓, ?)

l_2-norm. MSE

e.g. B. $z \sim N(0, \Sigma)$

$l(x,y;\theta) = [y - f(x;\theta)]^T \Sigma^{-1} [y - f(x;\theta)]$

e.g. C. $z \sim \text{Laplace}(0, b)$

$l(x,y;\theta) = \text{const} + \frac{1}{b}|y - f(x;\theta)|$

$L(\theta) = \frac{1}{Nb}\sum_{n=1}^{N} |y^{(n)} - f(x^{(n)};\theta)|$ **L_1-norm. MAE**

For **classification**

$x \in \mathbb{R}^d$ input

$y \in \{e_1 \cdots e_c\}$ class label in one-hot coding.

令 $P_i = P(y = e_i | x)$

$P(y|x) = \prod_{i=1}^{c} P_i^{y_i}$ PMF.

我们用 $[f_i(x;\theta)]$ 估计 P_i

那么 $P(y|x)$ 近似为 $Q(y|x;\theta) = \prod_{i=1}^{c} f_i^{y_i}$

loss $l(x,y;\theta) = -\ln Q = -\sum_{i=1}^{c} y_i \ln f_i$ **categorial CE loss.**

为使 $0 < f_i < 1$

$\sum_{i=1}^{c} f_i = 1$, 使用 softmax 作为 output 的 ϕ.

Semantic Segmentation.

$\hat{=}$ pixelwise classification.

这里还是用 **categorial CE loss.**

因为他本质还是属于 classification.

$X = [x_{mn}]_{mn}$ $Y = [y_{mn}]_{mn}$ $\hat{Y} = [\hat{y}_{mn}]_{mn} = f(x;\theta)$

$$l(x,y;\theta) = \sum_{h=1}^{H}\sum_{w=1}^{W} \underbrace{\underbrace{[-y_{hw}^T \ln \hat{y}_{hw}(x;\theta)]}_{\text{one pixel}}}_{\text{image}}$$

假如类分部非常不均.

① weighted loss ② region-based loss.

⇓

Soft J and D.

$Y = [y_{mn}]_{mn}$ $\hat{Y} = [\hat{y}_{mn}]_{mn}$

$y_{mn} \in \{e_1, \cdots, e_c\}$ one hot coding.

$\hat{y}_{mn} \in \mathbb{R}^c$ class softmax output.

$\alpha = \sum_{m=1}^{M}\sum_{n=1}^{N} y_{mn} \odot \hat{y}_{mn} = [\alpha_i]$

$\beta = \sum_{m=1}^{M}\sum_{n=1}^{N} (y_{mn} + \hat{y}_{mn}) = [\beta_i]$

$J_i = \frac{\alpha_i + \epsilon}{\beta_i - \alpha_i + \epsilon}$.

$l(X,Y;\theta) = 1 - \frac{1}{c}\sum_{i=1}^{c} J_i$

那么，什么是 Training ？

*) training set $D = \{\underline{x}^{(n)}, \underline{y}^{(n)}, 1 \le n \le N\}$

*) cost function $L(\underline{\theta})$

$\Longrightarrow$ task: $\min_{\underline{\theta}} L(\underline{\theta})$ ，就是 Training.

对于 DL，也就是 DNN，它是：

$$\nabla L(\underline{\theta}) = \frac{1}{N}\sum_{n=1}^{N} \nabla l(\underline{x}^{(n)}, \underline{y}^{(n)}; \underline{\theta})$$

gradient descent or variants 梯度下降.

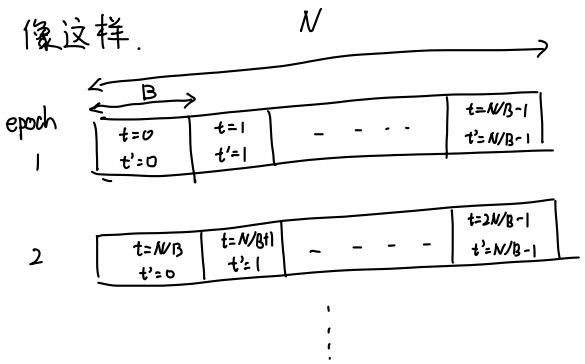
Update: $\underline{\theta}^{t+1} = \underline{\theta}^{t} - \gamma^{t} \nabla L(\underline{\theta}^{t})$

γ^t step size， $\underline{\theta}^0$ 初始化.

但实际上，我们常把 N 这样的 dataset 分为很多个 minibath B.

在每个 B 中更新 $\underline{\theta}$，计算 $\nabla L(\underline{\theta})$

像这样.



$$\underline{\theta}^{t+1} = \underline{\theta}^{t} - \gamma^{t} \nabla L(t;\underline{\theta})\big|_{\underline{\theta}=\underline{\theta}^{t}}$$

$$L(t;\underline{\theta}) = \frac{1}{B}\sum_{n=t'B+1}^{(t'+1)B} l(\underline{x}^{(n)}, \underline{y}^{(n)}; \underline{\theta})$$

$t = 0, 1, \cdots\cdots$

$t' = \mathrm{mod}(t, \frac{N}{B}) \in \{0, 1, \cdots \frac{N}{B}-1\}$

但是 $\nabla L(t;\underline{\theta})$ 总是比 $\nabla L(\underline{\theta})$ more noisy.

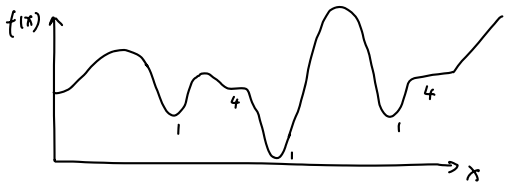
$\Longrightarrow$ stochastic gradient decent.

用 $\hat{\underline{\theta}}_k$ 来解决，具体没搞懂，在 163 页.

Training 其实是一个 optimazition 问题，

他就不断地在迭代 $\underline{\theta}$.

下面罗列一些他会遇到的困难.



1. local min.
2. local max.
3. saddle points
4. plateau, region of constant $f(x)$

4 由很多 3 组成

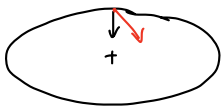
我们找梯度降落，其实是找一个 local min 点，

他不一定是最优的解，但是一个有意义的伏解.

D1) stochastic gradient.

在实际问题中，我们使用了 minibatch B.

而 $B \ll N$，这会导致梯度降落方向不准确.

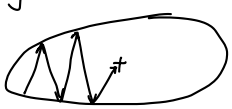


B 越大这个就越准确.

但 B 越小，每个 epoch 就越快.

D2) ill conditioning

这个是指.



这种情况，使 convergence 变慢.

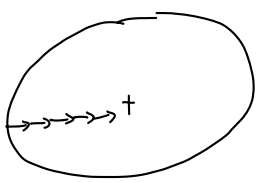
D3) saddle point, / plateau.

$\nabla L = 0$， Update 停止了.

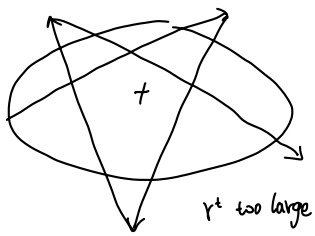
实际上当 $\nabla L \doteq 0$ 时，Update 的每一步非常小，

convergence 超级慢.

D4) Sensitive to the choice of the step size (γ^t)



γ^t too small



γ^t too large.

D5) local min.

字面意思，找到的是 local min.

不是 global min.

D6) vanishing gradient.

这个很复杂，我没理解.

我直接抄这里.

Backpropagation of the error vector.

$$\underline{\delta}_L^T = \frac{\partial L(\underline{\theta})}{\partial \underline{a}_L} \in \mathbb{R}^{1\times M_L}.$$

$$\underline{\delta}_1^T = \underbrace{\underline{\delta}_L^T \cdot \underline{\underline{J}}_{\underline{a}_L}(\underline{a}_{L-1}) \cdot \cdots \underline{\underline{J}}_{\underline{a}_2}(\underline{a}_1)}_{L \text{ Layers}}$$

$$\underline{\underline{J}}_{\underline{a}_{L+1}}(\underline{a}_L) = \underline{\underline{J}}_{\underline{a}_{L+1}}(\underline{x}_L)\cdot \underline{\underline{J}}_{\underline{x}_L}(\underline{a}_L) = \underline{\underline{W}}_{L+1}\, \mathrm{diag}(\phi_L'(\underline{a}_L))$$

*) If $\|\underline{\underline{J}}_{\underline{a}_{L+1}}(\underline{a}_L)\| < 1 \ \forall L$, Hence $\|\underline{\delta}_L\| \xrightarrow{L\to 1} 0$

vanishing gradient → no update of θ^t

→ stop learning.

*) If $\|\underline{\underline{J}}_{\underline{a}_{L+1}}(\underline{a}_L)\| > 1 \ \forall L$ Hence $\|\underline{\delta}_L\| \xrightarrow{L\to 1} \infty$

exploding gradient → divergence.

*) no problem for shallow NNs (L↓)

big problem for deep NNs (L↑)

*) less serious for last layers. (L↑)

more serious for first layers. (L↓)

sigmoid or tanh bad for DNN, due to D6

ReLu less serious for D6.

下面我们来解决上面提到的问题.

① Momentum，解决 D1, D2

$$\Delta\underline{\theta}^t = \beta\cdot\Delta\underline{\theta}^{t-1} - \gamma^t \nabla_{\underline{\theta}} L(t;\underline{\theta})\big|_{\theta=\theta^t}$$

$$\underline{\theta}^{t+1} = \underline{\theta}^t + \Delta\underline{\theta}^t \qquad 0\le\beta<1$$

$\beta=0 \rightarrow$ SGD.

这是一种 recursive 的思维. 类似于 IIR filter.

Nesterov momentum.

$$\Delta\underline{\theta}^t = \beta\cdot\Delta\underline{\theta}^{t-1} - \gamma^t \nabla_{\underline{\theta}} L(t;\underline{\theta})\big|_{\underline{\theta}=\underline{\theta}^t+\beta\cdot\Delta\underline{\theta}^{t-1}}$$

② Learning rate schedule. 解决 D4.

我们很难找到一个 $\gamma^t=\gamma$.

所以我们

1、Static schedule.

γ^t 与 $\underline{\theta}^t$ 无关，但与 t 有关.

e.g. 

需要选一个 γ^0.

2、Adaptive schedules.

γ^t 与 $\underline{\theta}^t$ 相关，很复杂.

③ Input and batch normalization. D2,

1、介绍一种 Input

zero-mean, unit-variance normalization

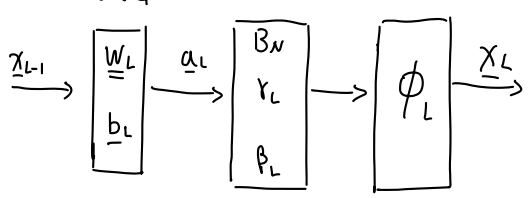
$$x_i(n) \leftarrow \frac{x_i(n)-\mu_i}{\sigma_i}$$

解决 D2, faster convergence.

2、Batch.

这是对于任意 hidden layer 的 normalization.

对每一个 B.



BN Layer. ϵ, a small positive number. to avoid division by zero.

$$[\underline{a}_L(n)]_i \leftarrow \gamma_{L,i}\frac{[\underline{a}_L(n)]_i - \mu_{L,i}}{\sqrt{\sigma_{L,i}^2+\epsilon}} + \beta_{L,i}, \quad 1\le n\le B,\ 1\le i\le M_L.$$

① $\mu_{L,i} = \frac{1}{B}\sum_{n=1}^{B}[\underline{a}_L(n)]_i$ and $\sigma_{L,i}^2 = \frac{1}{B-1}\sum_{n=1}^{B}([\underline{a}_L(n)]_i-\mu_{L,i})^2$

④ Parameter initialization. D5.

初始θ选的好，local min. 也可能是 global min.

1. zero initialization. $\underline{\theta}^0 = \underline{0}$

2. random initialization of $\underline{W}_l$

还有两种，太复杂了，先不写了.

⑤ Improved model.

改变 DNN 的结构去让 opt. 更简单.

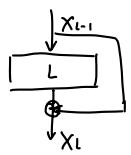
A) activation function.

ReLU > sigmoid

leaky ReLU > ReLU.

B) Skip connections, shortcuts, residual network (ResNet)

over one layer



下面我们要学习，如何化解 overfitting

我们用 model capacity 来表示一个 model 的学习能力.

too low model capacity ⟶ underfitting.

too high model capacity ⟶ overfitting.

⟹ find a right model, not too complex, not too simple.

Regularization

1. Weight norm penalty.

change on cost function.

$$L(\underline{\theta}) \Longrightarrow L_r(\underline{\theta}) = L(\underline{\theta}) + \sum_{l=1}^{L} \lambda_l P(\underline{W}_l)$$

$\lambda_l \geq 0$: regularization parameter.

$P(\underline{W}_l) \geq 0$: penalty.

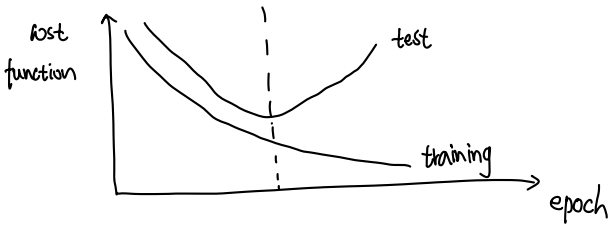
a) L_2 -regularization. l_2-norm of vec $(\underline{W}_l)$

$$P(\underline{W}_l) = \|\text{vec}(\underline{W}_l)\|_2^2 = \sum_i \sum_j W_{ij}^2$$

b) L_1 - regularization. l_1-norm of vec $(\underline{W}_l)$

$$P(\underline{W}_l) = \|\text{Vec}(\underline{W}_l)\|_1 = \sum_i \sum_j |W_{l\,ij}|$$

2. Early stopping.



stop the training before the test error increases

3. Data augumentation.

change on dataset.

training with ∞ many training data ⟶ no overfitting.

生成合成的一些真实 dataset. 去扩展 training dataset.

good for classification

difficult for regression.

4. Ensemble learning

change on dataset / model / cost function / optimizer.

用不同的模型 (independent) train 相同的任务.

再把结果结合再一起.

regression: average

classification: voting.

5. Dropout.

change on model.

在 Training 时，对于每个 B，

随机的扔掉一些 neuron.

with probability d_l. (dropout rate)

对于一个相同的 network 的不同 dropout

这是一个 model 当做的 ensemble learning.

在 Inference, no dropout.

6. Hyperparameter optimization.

在 ML 中不被 adapted 的参数

要做他的 optimization.

需要一个 Validation set 专门来做

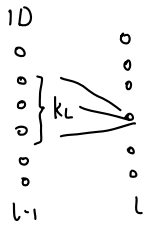
$f(\underline{x};\underline{\theta}) \longrightarrow f(\underline{x};\underline{\theta},\underline{\eta})$

Training set $\longrightarrow \underline{\theta}$

Validation set $\longrightarrow \underline{\eta}$

Test set $\longrightarrow$ test error $\longrightarrow$ avoid overfitting of $\underline{\theta}$ / avoid overfitting of $\underline{\eta}$

那么 C layer 的优势都有什么.



P_1) sparse connections

focus on kernel.

(local input)

P_2) Parameter sharing.

the same kernel for all output neurons.

→ 降低了复杂度. reduce overfitting. N_p, N_x

P_3) translation - equivalant.

translation of $\underline{x}_{l-1}$ → same for $\underline{x}_l$

P_4) one channel map continus one feature.

P_5) $M_L = M_{L-1} - k_L + 1$ become smaller.

→ 对于 C layer 还可以有一些提升.

下面我们认识一种新的. 特殊 NN.

Convolutional neural networks (CNN)

在 DNN 中, 参数的数量随 layer 增长极快.

是二次增长的. $W_L \in \mathbb{R}^{M_L \times M_{L-1}}$

CNN 可以一定程度解决这样的问题. → mainly convolutional layers.

*) good for local feature learning

*) efficient, low complexity.

2D input → 4D kernel tensor → 3D output tensor

C. layer

input tensor $\underline{x}_{L-1}$ $M_{L-1} \times N_{L-1} \times D_{L-1}$ → $M_{L-1} \times N_{L-1}$

output tensor $\underline{x}_L$ $M_L \times N_L \times D_L$ → $M_L \times N_L$

$\underline{x}_L = \phi_L(\underline{A}_L)$

kernel tensor $\underline{W}_L$: $K_L \times K_L \times D_{L-1} \times D_L$

bias vector $\underline{b}_L$ $D_L \times 1$

activation tensor A_L $M_L \times N_L \times D_L$

$$[\underline{A}_L]_{hwo} = \sum_{i=1}^{k_L} \sum_{j=1}^{K_L} \sum_{d=1}^{D_{L-1}} [\underline{W}_L]_{ijdo} [\underline{X}_{L-1}]_{h+i-1,\, w+j-1,\, d} + [\underline{b}_L]_o$$

convolution in CNN

in CNN.

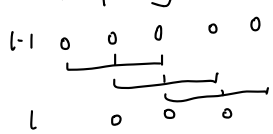
$N_p = K_L^2 D_{L-1} D_L + D_L \doteq k_L^2 D_{L-1} D_L$

$N_x = M_L N_L D_L \cdot K_L K_L D_{L-1} \doteq M_L M_L \cdot N_p$.

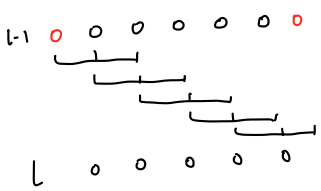
$N_x >> N_p$

1. Padding.

no padding $K_L = 3$

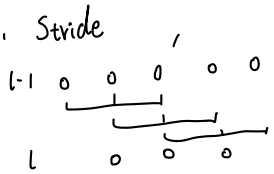


padding with P=1

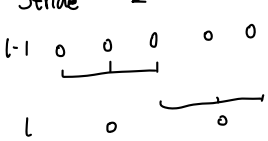


→ larger output size.

2. Stride $K_L = 3$

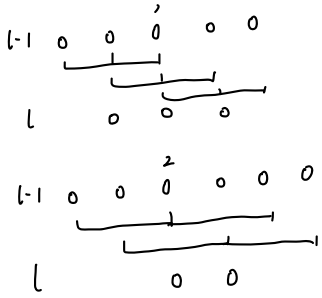


Stride 2



→ reduce the size.

3. Dilated convolution.



standard

$M_l = M_{l-1} - K_l + 1$

modified

$\left\lfloor \frac{M_{l-1} + 2P - (K_l - 1)D - 1}{S} \right\rfloor + 1$

对于 C.layer的 Normalization.

① weight ② activation.

CNN: *) spatial dimention H x W

*) channel depth D.

*) minibatch B.

Encoder, backbone

Head, Decoder

具体. ← 7-20.

CNN 是一个 Encoder + head / decoder.

对于 C. layer

reshapping → desen layer.

$\underline{\underline{X}}_l \longrightarrow \underline{X}_l$

1) Flatten layer

$\underline{\underline{X}} \in \mathbb{R}^{H \times W \times D} \longrightarrow \text{vec}(\underline{\underline{X}}) \in \mathbb{R}^{HWC}$

no reduction of number of element

high complexity

2) Global average pooling (GAP) layer.

$\underline{\underline{X}} = [x_{ijd}] \in \mathbb{R}^{M \times N \times D} \xrightarrow{GAP} \left[\frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} x_{ijd} \right] \in \mathbb{R}^{D}$

reduce number.

much less neurons in dense layer

used in CNNs.

- Downsampling Layer. / pooling.

size $H_l \times W_l \longrightarrow X_l$ by stride

lower level feature → high level feature

*) max pooling $P = \max$

*) mean pooling $P = \text{mean}$

*) l_2-norm pooling $P = l_2\text{-norm}$

- upsampling layer

increase the size X_l.

reconstraction of the original image.

deconvolution

至此现在.

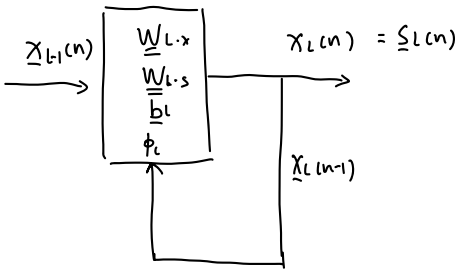
NN feedback

DNN, CNN. feedforward.

下一页开始 RNN.

by the way. RNN 是 feedback.

R layer



通常 $W_{l,x}$ full matrix

$W_{l,s}$ diagonal

$s_l(0)$ 是需要初始化的

$$a_l(n) = W_{l,x}\, x_{l-1}(n) + \underbrace{W_{l,s} \cdot s_l(n-1)}_{\text{feedback.}} + b_l$$

RNN, with at least on R layer

Training. 使用. SGD : $\theta^{t+1} = \theta^t - \gamma^t \nabla L(t;\theta)\big|_{\theta=\theta^t}$

还需要一个对于 R layer 的 parameter.

在 Train RNN 时, 常常发生 vanishing gradient.

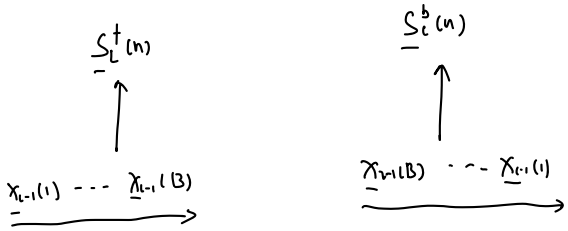
B越小, 就越不准. B大 ↗

对此我们采用. long short term memory (LSTM)

BRNN

$x_l(n)$ 是对 backward 与 forward 的结合.

$$x_l(n) = \begin{bmatrix} s_l^f(n) \\ s_l^b(n) \end{bmatrix}$$



$\Rightarrow x_l(n)$ depends on all $x_{l-1}(1)$ --- $x_{l-1}(B)$

$i_l(n)$ $o_l(n)$ $f_l(n)$

三种门, 控制 input, output, forget.

$$s_l(n) = f_l(n) \odot s_l(n-1) + i_l(n) \odot \phi_l(W_{l,1x}\, x_{l-1}(n) + W_{l,so}\, x_l(n-1) + b_{ls})$$

$$x_l(n) = o_l(n) \odot \phi_l(s_l(n))$$

基础部分就到这里了.

下面是一些超越的东西.

Attention and Transformer

Cross-correlation = Attention.

Autocorrelation = self-Attention.

$X \in \mathbb{R}^{H\times W\times C}$ ← from max pooling or 2 dense layers.

$$[\tilde{X}]_{hwc} = [X]_{hwc} \cdot [M^c]_c$$

↳ channel attention

$$[\tilde{X}]_{hwc} = [X]_{hwc} \cdot [M^s]_{hw}$$

M^s ← from max pooling or 1 conv. layer

↳ spatial attention.

$M^c \in (0,1)^C$ $M^s \in (0,1)^{H\times W}$

Transformer. 利用 sequences of words.

seq 2 seq model based on attention.

理解与生成语言. 不用 recusive. 但需要大量 dataset.

1. Language transformers

*) Tokenization. 一个 text 分成多个单元

*) token embedding tokens → numbers

*) transformer

Self-attention: 学习 input 中, 两两 token 之间的关系.

2. Vision transformers.

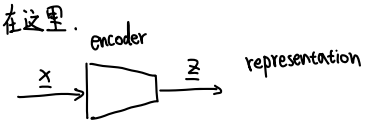
pixel = token.

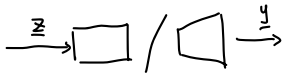
Self-supervised learning

Pretraining：用 other dataset 先 train 一下

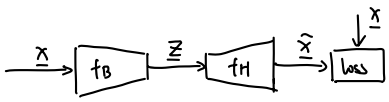
Finetuning：用一个小的 labeled dataset from DT 去修正一下.

在这里.





Autoencoder

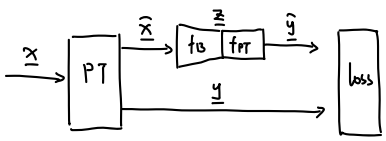


focus on z but not x̂

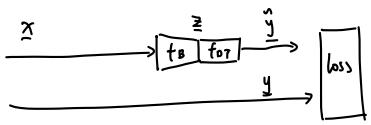
Pretext task.

人工生成一个 supervised task.

对 x(n) 生成 y(n)



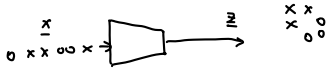
再用这个 fB 去解决 DT



Constrastive learning

对比相同的 samples. 无 y, ŷ, 无 task

只有 encoder



Fondation learning

用大量 data 做 pretraining.

一个超大模型

Transfer learning

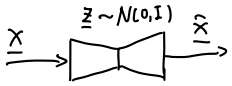
通过对解决 旧 task 的模型 做出改变.

使他可以解决新问题.

Generative model. 生成模型.

z → x

1. VAE. variational autoencoder.

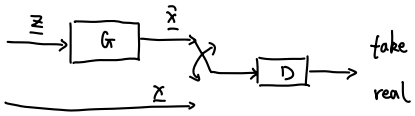


我们知到 z 的分布.

就可以由此通过 decoder 再生成 x̂

2. Generative adversarial network (GAN)

这是一个 Generative (G) 与 Discriminator (D) 的竞争游戏.



3. Diffusion model. 没懂.